

FAQ Higiene de Estado de Sesión — Diagramas de Flujo

➤ Traducción de `faq_higiene_estado_sesion.md` a simbología ISO 5807 / ANSI X3.5: cadena de integridad, guardia MCP, veredictos de `check/clean` y eslabón semántico de bitácoras

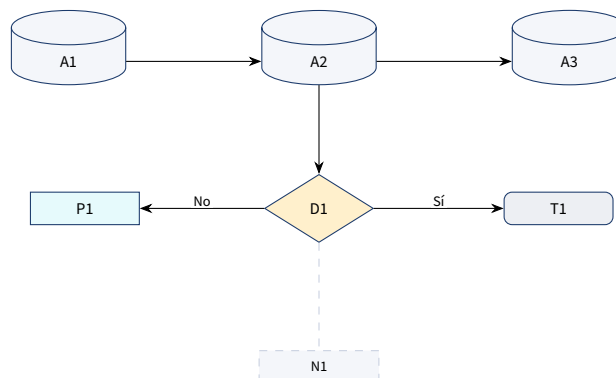
🚩 Leyenda de símbolos—ISO 5807 / ANSI X3.5

Óvalo = **terminador** (inicio/fin) · Rectángulo = **proceso** (acción determinista) · Rombo = **decisión** (ramas Sí/No) · Paralelogramo = **entrada/salida** (lectura/escritura de archivos) · Cilindro = **almacenamiento online** (`session_log_*.jsonl`, `run_state.json`) · Rectángulo con base ondulada = **documento** (`Sessions/*.md`) · Rectángulo punteado gris = **nota explicativa** (sin acción). Flujo: arriba→abajo, izquierda→derecha.

🔗 Cada símbolo lleva una **etiqueta** (T/P/D/E/A/N + número, según su tipo); su significado se expande en la tabla *Leyenda de etiquetas* inmediatamente bajo cada diagrama.

1 🔗 Cadena de integridad del log append-only

El log `session_log_<run>.jsonl` es la fuente de verdad inmutable: cada evento encadena (`prev`) el hash SHA-256 de la línea anterior, la secuencia `seq` debe ser contigua y la marca "`append_only`": `true` es exigida por `integrity`. Cualquier edición, borrado o reordenamiento rompe la cadena.



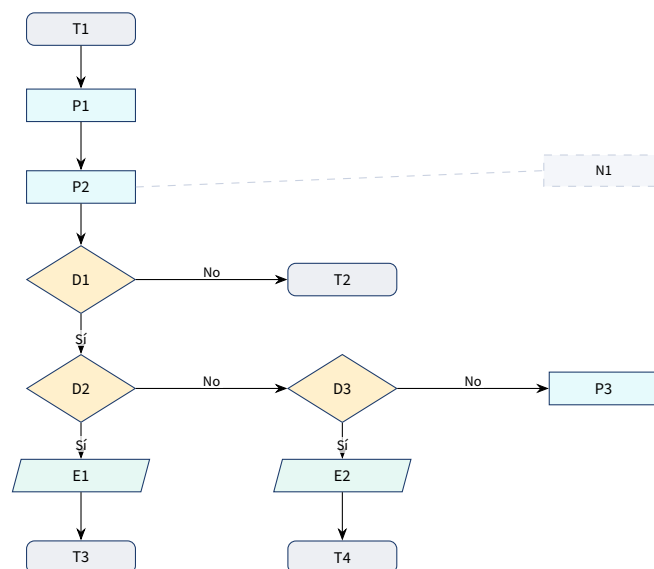
📋 Leyenda de etiquetas

Etiqueta	Significado
A1	Evento 1: primer evento del log, seq=1 y sin prev.
A2	Evento 2: seq=2, prev = hash SHA-256 del evento anterior.
A3	Evento n (último): seq=n, prev = hash del evento n-1 (encadenamiento).
D1	¿integrity OK? Verifica prev=hash, seq contigua y append_only.
T1	Log íntegro (tamper-evident): ninguna edición pasó desapercibida.
P1	Manipulación detectable: un desajuste de hash o de secuencia delata edición, borrado o reordenamiento.
N1	Es el primitivo de <i>hash-chain</i> , pero sin merkle trees, sin consenso, sin descentralización ni incentivos: tamper-evident sí; blockchain propiamente, no.

→ **Contraste con** `run_state.json`: el log es *criptográficamente encadenado* (toda edición es detectable), mientras que la vista es un JSON plano que se sobrescribe sin rastro. Por eso el log es **fuentes de verdad** y `run_state*.json` una **vista derivada**.

2 🛡️ Guardia MCP: ¿el estado leído pertenece a la corrida actual?

La guardia usa la comparación de *mtime antes/después* de la corrida. Es la única señal que vincula temporalmente el archivo al subproceso sin conocer el `run_id` de antemano.

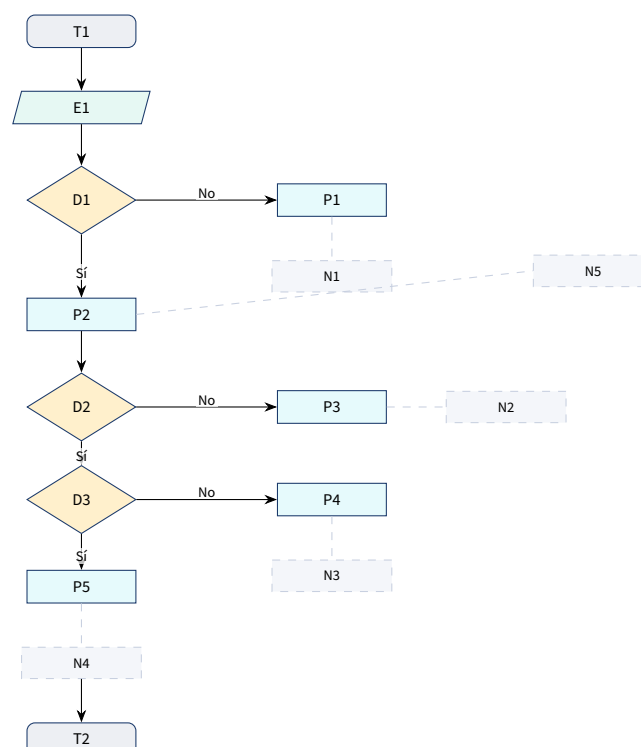


Leyenda de etiquetas

Etiqueta	Significado
T1	Corrida MCP lanzada.
P1	Toma <code>mtime_before = mtime(run_state.json)</code> antes de la corrida (None si el archivo no existe).
P2	Ejecutar el flujo (subprocess).
D1	¿Existe <code>run_state.json</code> al término de la corrida?
T2	<code>state_data = {}</code> (sin vista): el flujo no escribió estado.
D2	¿ <code>mtime_before = None</code> ? (el archivo no existía antes de la corrida).
E1	Leer el estado: fue <i>creado</i> durante esta corrida.
T3	Vista pertenece a la corrida.
D3	¿ <code>mtime_ahora > mtime_before</code> ? (el estado sí fue reescrito en esta corrida).
P3	No usar: vista huérfana (<code>mtime</code> de otra corrida).
E2	Leer el estado: fue <i>reescrito</i> durante esta corrida.
T4	Vista pertenece a la corrida.
N1	<code>flujo/fin</code> no sirve de guardia: es un evento de <i>ciclo de vida</i> que aparece en los dos casos opuestos (huérfano: el run de hace 1 h terminó y dejó su estado; legítimo: el flujo recién lanzado terminó bien). Con el criterio “¿hay flujo/fin? → descartar” se descartaría el estado legítimo que el MCP lee <i>después</i> de que el flujo termina.

3 estado_sesion.py: veredictos y limpieza de la vista

Principio conservador: **“borro solo con certeza”**. huérfano → borra; vigente y `no_verificable` → conserva; corrupto → conserva y reporta.



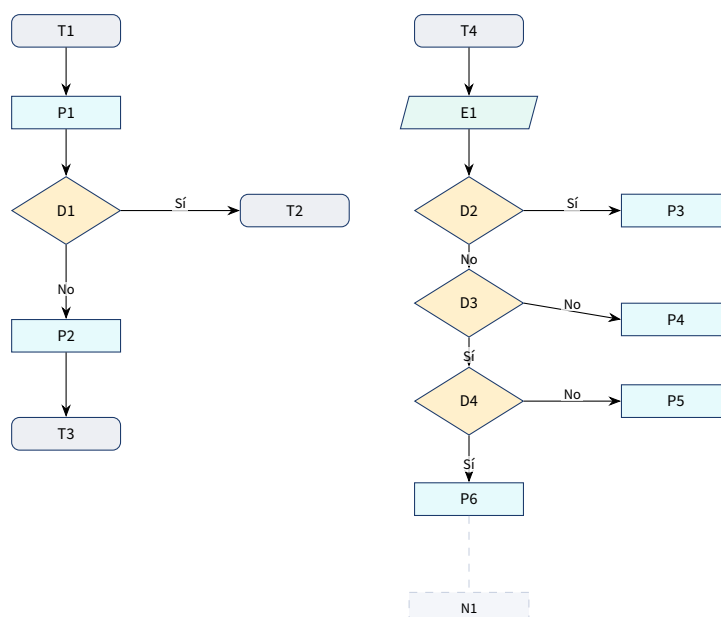
Leyenda de etiquetas

Etiqueta	Significado
T1	check(run_id): inicia la verificación de la vista derivada.
E1	Buscar el log session_log_<run>.jsonl del run_id.
D1	¿Existe el log para el run_id?
P1	Veredicto: no_verificable.
N1	Sin log → sin certeza de si la corrida terminó. clean NO actúa (ni lo lista).
P2	integrity: prev = hash $\wedge$ seq contigua $\wedge$ append_only.
D2	¿La cadena está íntegra?
P3	Veredicto: corrupto.
N2	Conservar y reportar (incrementa anomalías).
D3	¿El último evento es flujo/fin?
P4	Veredicto: vigente.
N3	Conservar (en curso, error o vacío).
P5	Veredicto: huérfano.
N4	Único caso que clean borra (unlink).
T2	Vista eliminada.
N5	modelo_obsoleto: solo <i>advertencia</i> informativa (deepseek-v4-pro); no altera veredicto ni incrementa anomalías.

El modelo obsoleto es una señal de atención (“reanudar con este modelo fallaría”), no un criterio de limpieza: en el caso real detectado se borró por huérfano, no por el modelo.

4 Eslabón semántico: bitacoras.py (nueva y check)

El subcomando nueva crea la bitácora canónica (nunca sobrescribe); check clasifica cada bitácora por su fecha relativa a FECHA_PLANTILLA (2026-09-11).



 Leyenda de etiquetas**Etiqueta Significado***Columna A — “nueva”*

T1	nueva -tema: crea la bitácora canónica de la sesión.
P1	Slug ASCII del tema → archivo <code>Sessions/YYYY-MM-DD_<slug>.md</code> .
D1	¿Ya existe el archivo?
T2	<code>ya_existia</code> (nunca sobrescribe).
P2	Escribir las 5 secciones: Tema · Contexto · Decisiones (usuario) · Actividades · Pendientes.
T3	Lista para check.

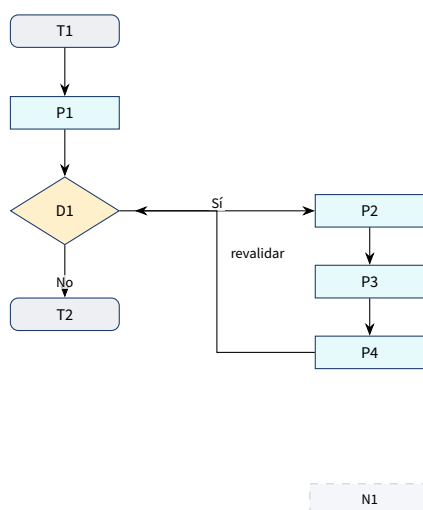
Columna B — “check”

T4	check: clasifica cada bitácora según su fecha relativa a <code>FECHA_PLANTILLA</code> .
E1	Listar <code>Sessions/* .md</code> .
D2	¿Fecha del nombre < <code>FECHA_PLANTILLA</code> (2026-09-11)?
P3	Legado: ok forzado <code>True</code> (informativo, no se reescribe).
D3	¿Secciones completas (hoy/reciente)?
P4	Anomalía accionable: afecta veredicto y <code>exit code</code> .
D4	¿hay_bitacora para hoy?
P5	Veredicto: “atencion” (día sin bitácora).
P6	Veredicto: ok.
N1	Legado (antes de 2026-09-11): escritas con otra convención (ej. las 33 antiguas sin Decisiones (usuario)). Reescribirlas sería falsificar historia.

En la columna derecha, “hoy” o “reciente” sin secciones completas es una anomalía real a corregir *antes* de cerrar; la salida cruza con `sesion_hoy`, booleano que check obtiene filtrando por el prefijo del nombre (`startswith("YYYY-MM-DD")`).

5 Cierre de sesión: remediación obligatoria (edge case de la directiva)

Si al cierre check reporta *falta de bitácora del día* o *secciones vacías* (hoy/reciente), la sesión no debe cerrar así: la recuperación es correctiva y el defecto no es cosmético, porque el contexto semántico es irre recuperable una vez cerrada la sesión (no se regenera desde ningún log).



Leyenda de etiquetas

Etiqueta	Significado
T1	Cierre de sesión.
P1	Ejecutar <code>bitacoras.py check</code> : verifica la bitácora del día y el llenado de secciones.
D1	¿Falta la bitácora del día o hay secciones vacías (hoy/reciente)?
T2	Cerrar sesión (sin defectos de bitácora).
P2	nueva -tema (si falta la bitácora del día).
P3	Rellenar las secciones semánticas.
P4	Revalidar con <code>check</code> .
N1	Exento: las bitácoras legado (antes de 2026-09-11) son informativas y no se reescriben.

Concepto clave

💡 **Información que reside exclusivamente en Sessions/:** el **contexto semántico**—el POR QUÉ de las decisiones (sobre todo las del usuario), lo que se descartó y por qué, intenciones y matices, reglas y aprendizajes derivados, y pendientes acordados con su contexto. Bajo nivel = datos reproducibles (log + run_state); alto nivel = significado (Sessions/). Sin bitácora, esa información se pierde *definitivamente*.